

21 Text and Binary file pointers - Zero to Hero questions list

July 18, 2026

1 Text and Binary File Pointers in C — Practice Questions (50 Questions with Hints Only)

The questions are arranged from **Beginner** → **Intermediate** → **Advanced**. They focus on **file pointers** (`FILE *`), **text files**, **binary files**, **file positioning functions**, and **random file access**.

2 Beginner Level (1–15)

2.0.1 1. Create a text file and write a message into it.

Hint: Use `FILE *fp`, `fopen()` in "w" mode, and `fprintf()`.

2.0.2 2. Read and display the contents of a text file.

Hint: Use "r" mode and `fgetc()`.

2.0.3 3. Write multiple lines to a text file.

Hint: Use `fputs()` or `fprintf()`.

2.0.4 4. Read a text file line by line.

Hint: Use `fgets()` until `NULL`.

2.0.5 5. Copy one text file into another.

Hint: Read character by character and write to the destination file.

2.0.6 6. Append data to an existing text file.

Hint: Open the file using "a" mode.

2.0.7 7. Count the number of characters in a text file.

Hint: Read until EOF.

2.0.8 8. Count the number of words in a text file.

Hint: Detect spaces, tabs, and newlines.

2.0.9 9. Count the number of lines in a text file.

Hint: Count occurrences of '\n'.

2.0.10 10. Display the current position of the file pointer.

Hint: Use `ftell()`.

2.0.11 11. Move the file pointer to the beginning of a file.

Hint: Use `rewind()`.

2.0.12 12. Move the file pointer to a specific byte position.

Hint: Use `fseek()`.

2.0.13 13. Determine the size of a text file.

Hint: Move to the end using `fseek()` and use `ftell()`.

2.0.14 14. Check whether a file opened successfully.

Hint: Verify if `FILE *` is `NULL`.

2.0.15 15. Display the file position before and after reading characters.

Hint: Use `ftell()` after each read.

3 Intermediate Level (16–35)

3.0.1 16. Replace every lowercase letter with uppercase in a text file.

Hint: Copy modified data into a temporary file.

3.0.2 17. Replace every uppercase letter with lowercase.

Hint: Process one character at a time.

3.0.3 18. Count vowels, consonants, digits, and special characters in a file.

Hint: Classify each character while reading.

3.0.4 19. Remove blank lines from a text file.

Hint: Skip empty lines during copying.

3.0.5 20. Remove extra spaces from a text file.

Hint: Keep track of consecutive spaces.

3.0.6 21. Merge two text files into a third file.

Hint: Copy both files sequentially.

3.0.7 22. Compare two text files.

Hint: Compare characters until EOF.

3.0.8 23. Reverse the contents of a text file.

Hint: Use `fseek()` or store the contents in memory.

3.0.9 24. Display the last N characters of a file.

Hint: Use `fseek()` relative to the end of the file.

3.0.10 25. Display the last N lines of a file.

Hint: Traverse backwards or store line positions.

3.0.11 26. Find the longest line in a text file.

Hint: Read using `fgets()` and compare lengths.

3.0.12 27. Find the shortest line in a text file.

Hint: Keep track of the minimum length.

3.0.13 28. Search for a word in a text file.

Hint: Read line by line and tokenize.

3.0.14 29. Replace a word throughout a text file.

Hint: Use a temporary file for rewriting.

3.0.15 30. Count the frequency of each alphabet in a text file.

Hint: Use an array of 26 counters.

3.0.16 31. Store student records in a binary file.

Hint: Use `fwrite()` with structures.

3.0.17 32. Read student records from a binary file.

Hint: Use `fread()`.

3.0.18 33. Append new records to a binary file.

Hint: Open the file using "ab" mode.

3.0.19 34. Count the number of records in a binary file.

Hint: Divide the file size by the size of one structure.

3.0.20 35. Display binary file records one by one.

Hint: Continue reading until `fread()` returns 0.

4 Advanced Level (36–50)

4.0.1 36. Search for a student record in a binary file using roll number.

Hint: Compare each record while reading.

4.0.2 37. Update a student's marks in a binary file.

Hint: Use `fseek()` to move to the desired record.

4.0.3 38. Delete a record from a binary file.

Hint: Copy all records except the target into a new file.

4.0.4 39. Modify a specific record using random access.

Hint: Calculate the byte offset using `record_number * sizeof(struct)`.

4.0.5 40. Read the Nth record directly from a binary file.

Hint: Use `fseek()` with the correct offset.

4.0.6 41. Swap two records inside a binary file.

Hint: Read both records, exchange them, and write them back.

4.0.7 42. Sort records stored in a binary file by roll number.

Hint: Read all records into memory, sort, and rewrite.

4.0.8 43. Build an index for binary file records.

Hint: Store record positions along with key values.

4.0.9 44. Demonstrate the difference between text and binary files.

Hint: Write the same data to both formats and compare size and contents.

4.0.10 45. Implement random access for employee records.

Hint: Access records directly using `fseek()` and `ftell()`.

4.0.11 46. Build a Student Database using binary files.

Hint: Implement:

- Add Record
 - Display Records
 - Search Record
 - Update Record
 - Delete Record
-

4.0.12 47. Build an Employee Payroll System using binary files.

Hint: Store employee structures and update salary records directly.

4.0.13 48. Build a Library Management System using text and binary files.

Hint: Store book information permanently and allow issue/return operations.

4.0.14 49. Build a Banking Management System using random-access binary files.

Hint: Implement:

- Create Account
- Deposit
- Withdraw

- Balance Inquiry
 - Update Customer Details
 - Transaction History
-

4.0.15 50. Build a Mini Database Engine using text and binary file pointers.

Hint: Divide the program into functions such as:

- Create Database
- Open Database
- Insert Record
- Search Record
- Update Record
- Delete Record
- Sort Records
- Display All Records
- Backup Database
- Restore Database
- Exit